JavaScript is disabled on your browser.

**Java SE 27 & JDK 27**

# Class Method

java.lang.Object
  java.lang.reflect.AccessibleObject
    java.lang.reflect.Executable
      java.lang.reflect.Method

**All Implemented Interfaces:**

AnnotatedElement, GenericDeclaration, Member

---

```
public final class Method
extends Executable
```

A `Method` provides information about, and access to, a single method on a class or interface. The reflected method may be a class method or an instance method (including an abstract method).

A `Method` permits widening conversions to occur when matching the actual parameters to invoke with the underlying method's formal parameters, but it throws an `IllegalArgumentException` if a narrowing conversion would occur.

**Since:**

1.1

**See Also:**

Member,
Class,
Class.getMethods(),
Class.getMethod(String, Class[]),
Class.getDeclaredMethods(),
Class.getDeclaredMethod(String, Class[])

## *Field Summary*

**Fields declared in interface Member**

DECLARED, PUBLIC

## *Method Summary*

All Methods | Instance Methods

| Modifier and Type | Method | Description |
|---|---|---|
| boolean | **equals**(Object obj) | Compares this Method against the specified object. |

| | | |
|---|---|---|
| AnnotatedType | **getAnnotatedReturnType**() | Returns an AnnotatedType object that represents the use of a type to specify the return type of the method/constructor represented by this Executable. |
| <T extends Annotation> T | **getAnnotation**(Class<T> annotationClas | Returns this element's annotation for the specified type if such an annotation is *present*, else null. |
| Annotation[] | **getDeclaredAnnotations**() | Returns annotations that are *directly present* on this element. |
| Class<?> | **getDeclaringClass**() | Returns the Class object representing the class or interface that declares the method represented by this object. |
| Object | **getDefaultValue**() | Returns the default value for the annotation member represented by this Method instance. |
| Class<?>[] | **getExceptionTypes**() | Returns an array of Class objects that represent the types of exceptions declared to be thrown by the underlying executable represented by this object. |
| Type[] | **getGenericExceptionTypes** | Returns an array of Type objects that represent the exceptions declared to be thrown by this executable object. |
| Type[] | **getGenericParameterTypes** | |

| | | |
|---|---|---|
| | | Returns an array of `Type` objects that represent the formal parameter types, in declaration order, of the executable represented by this object. |
| `Type` | **`getGenericReturnType`**`()` | Returns a `Type` object that represents the formal return type of the method represented by this `Method` object. |
| `int` | **`getModifiers`**`()` | Returns the Java language modifiers for the executable represented by this object. |
| `String` | **`getName`**`()` | Returns the name of the method represented by this `Method` object, as a `String`. |
| `Annotation[][]` | **`getParameterAnnotations`**`(` | Returns an array of arrays of `Annotations` that represent the annotations on the formal parameters, in declaration order, of the `Executable` represented by this object. |
| `int` | **`getParameterCount`**`()` | Returns the number of formal parameters (whether explicitly declared or implicitly declared or neither) for the executable represented by this object. |
| `Class<?>[]` | **`getParameterTypes`**`()` | Returns an array of `Class` objects that represent the formal |

| | | |
|---|---|---|
| | | parameter types, in declaration order, of the executable represented by this object. |
| `Class<?>` | **`getReturnType`**`()` | Returns a `Class` object that represents the formal return type of the method represented by this `Method` object. |
| `TypeVariable<Method>` | **`getTypeParameters`**`()` | Returns an array of `TypeVariable` objects that represent the type variables declared by the generic declaration represented by this `GenericDeclaration` object, in declaration order. |
| `int` | **`hashCode`**`()` | Returns a hashcode for this `Method`. |
| `Object` | **`invoke`**`(Object obj, Object... args)` | Invokes the underlying method represented by this `Method` object, on the specified object with the specified parameters. |
| `boolean` | **`isBridge`**`()` | Returns `true` if this method is a bridge method; returns `false` otherwise. |
| `boolean` | **`isDefault`**`()` | Returns `true` if this method is a default method; returns `false` otherwise. |
| `boolean` | **`isSynthetic`**`()` | Returns `true` if this executable is a synthetic |

| | | |
|---|---|---|
| | | construct; returns `false` otherwise. |
| `boolean` | **`isVarArgs`**`()` | Returns `true` if this executable was declared to take a variable number of arguments; returns `false` otherwise. |
| `void` | **`setAccessible`**`(boolean flag)` | Set the `accessible` flag for this reflected object to the indicated boolean value. |
| `String` | **`toGenericString`**`()` | Returns a string describing this `Method`, including type parameters. |
| `String` | **`toString`**`()` | Returns a string describing this `Method`. |

**Methods declared in class Executable**

`accessFlags`, `getAnnotatedExceptionTypes`, `getAnnotatedParameterTypes`, `getAnnotatedReceiverType`, `getAnnotationsByType`, `getParameters`

**Methods declared in class AccessibleObject**

`canAccess`, `getAnnotations`, `getDeclaredAnnotation`, `getDeclaredAnnotationsByType`, `isAccessible`, `isAnnotationPresent`, `setAccessible`, `trySetAccessible`

**Methods declared in class Object**

`clone`, `finalize`, `getClass`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

## *Method Details*

### setAccessible

```
public void setAccessible(boolean flag)
```

**Description copied from class: `AccessibleObject`**

Set the `accessible` flag for this reflected object to the indicated boolean value. A value of `true` indicates that the reflected object should suppress checks for Java language access control when it is used. A value of `false` indicates that the reflected object should enforce checks for Java language access control when it is used, with the variation noted in the class description.

This method may be used by a caller in class `C` to enable access to a `member` of `declaring class` `D` if any of the following hold:

- `C` and `D` are in the same module.
- The member is `public` and `D` is `public` in a package that the module containing `D` `exports` to at least the module containing `C`.
- The member is `protected static`, `D` is `public` in a package that the module containing `D` exports to at least the module containing `C`, and `C` is a subclass of `D`.
- `D` is in a package that the module containing `D` `opens` to at least the module containing `C`. All packages in unnamed and open modules are open to all modules and so this method always succeeds when `D` is in an unnamed or open module.

This method may be used by JNI code with no caller class on the stack to enable access to a `member` of `declaring class` `D` if and only if:

- The member is `public` and `D` is `public` in a package that the module containing `D` `exports` unconditionally.

This method cannot be used to enable access to private members, members with default (package) access, protected instance members, or protected constructors when the declaring class is in a different module to the caller and the package containing the declaring class is not open to the caller's module.

**Overrides:**

`setAccessible` in class `AccessibleObject`

**Parameters:**

`flag` - the new value for the `accessible` flag

**Throws:**

`InaccessibleObjectException` - if access cannot be enabled

**External Specifications**

Java Native Interface Specification

**See Also:**

`AccessibleObject.trySetAccessible()`,

MethodHandles.privateLookupIn(Class, MethodHandles.Lookup)

## getDeclaringClass

```
public Class<?> getDeclaringClass()
```

Returns the `Class` object representing the class or interface that declares the method represented by this object.

**Specified by:**

getDeclaringClass in interface Member

**Specified by:**

getDeclaringClass in class Executable

**Returns:**

an object representing the declaring class of the underlying member

## getName

```
public String getName()
```

Returns the name of the method represented by this `Method` object, as a `String`.

**Specified by:**

getName in interface Member

**Specified by:**

getName in class Executable

**Returns:**

the simple name of the underlying member

## getModifiers

```
public int getModifiers()
```

Returns the Java language modifiers for the executable represented by this object.

**Specified by:**

getModifiers in interface Member

**Specified by:**

getModifiers in class Executable

**Returns:**

the Java language modifiers for the executable represented by this object

**See *Java Language Specification*:**

8.4.3 Method Modifiers

**See Also:**

`Executable.accessFlags()`

## getTypeParameters

```
public TypeVariable<Method>[] getTypeParameters()
```

Returns an array of `TypeVariable` objects that represent the type variables declared by the generic declaration represented by this `GenericDeclaration` object, in declaration order. Returns an array of length 0 if the underlying generic declaration declares no type variables.

**Specified by:**

`getTypeParameters` in interface `GenericDeclaration`

**Specified by:**

`getTypeParameters` in class `Executable`

**Returns:**

an array of `TypeVariable` objects that represent the type variables declared by this generic declaration

**Throws:**

`GenericSignatureFormatError` - if the generic signature of this generic declaration does not conform to the format specified in *The Java Virtual Machine Specification*

**See *Java Language Specification*:**

8.4.4 Generic Methods

**Since:**

1.5

## getReturnType

```
public Class<?> getReturnType()
```

Returns a `Class` object that represents the formal return type of the method represented by this `Method` object.

**Returns:**

the return type for the method this object represents

## getGenericReturnType

```
public Type getGenericReturnType()
```

Returns a `Type` object that represents the formal return type of the method represented by this `Method` object.

If the return type is a parameterized type, the `Type` object returned must accurately reflect the actual type arguments used in the source code.

If the return type is a type variable or a parameterized type, it is created. Otherwise, it is resolved.

**Returns:**

a `Type` object that represents the formal return type of the underlying method

**Throws:**

`GenericSignatureFormatError` - if the generic method signature does not conform to the format specified in *The Java Virtual Machine Specification*

`TypeNotPresentException` - if the underlying method's return type refers to a non-existent class or interface declaration

`MalformedParameterizedTypeException` - if the underlying method's return type refers to a parameterized type that cannot be instantiated for any reason

**Since:**

1.5

## getParameterTypes

```
public Class<?>[] getParameterTypes()
```

Returns an array of `Class` objects that represent the formal parameter types, in declaration order, of the executable represented by this object. Returns an array of length 0 if the underlying executable takes no parameters. Note that the constructors of some inner classes may have an implicitly declared parameter in addition to explicitly declared ones. Also note that compact constructors of a record class may have implicitly declared parameters.

**Specified by:**

`getParameterTypes` in class `Executable`

**Returns:**

the parameter types for the executable this object represents

## getParameterCount

```
public int getParameterCount()
```

Returns the number of formal parameters (whether explicitly declared or implicitly declared or neither) for the executable represented by this object.

**Specified by:**

`getParameterCount` in class `Executable`

**Returns:**

The number of formal parameters for the executable this object represents

**Since:**

1.8

## getGenericParameterTypes

```
public Type[] getGenericParameterTypes()
```

Returns an array of `Type` objects that represent the formal parameter types, in declaration order, of the executable represented by this object. An array of length 0 is returned if the underlying executable takes no parameters. Note that the constructors of some inner classes may have an implicitly declared parameter in addition to explicitly declared ones. Compact constructors of a record class may also have implicitly declared parameters, but they are a special case and thus considered as if they had been explicitly declared in the source. Finally note that as a `modeling artifact`, the number of returned parameters can differ depending on whether or not generic information is present. If generic information is present, parameters explicitly present in the source or parameters of compact constructors of a record class will be returned. Note that parameters of compact constructors of a record class are a special case, as they are not explicitly present in the source, and its type will be returned regardless of the parameters being implicitly declared or not. If generic information is not present, implicit and synthetic parameters may be returned as well.

If a formal parameter type is a parameterized type, the `Type` object returned for it must accurately reflect the actual type arguments used in the source code. This assertion also applies to the parameters of compact constructors of a record class, independently of them being implicitly declared or not.

If a formal parameter type is a type variable or a parameterized type, it is created. Otherwise, it is resolved.

**Overrides:**

`getGenericParameterTypes` in class `Executable`

**Returns:**

an array of `Type`s that represent the formal parameter types of the underlying executable, in declaration order

**Throws:**

`GenericSignatureFormatError` - if the generic method signature does not conform to the format specified in *The Java Virtual Machine Specification*

TypeNotPresentException - if any of the parameter types of the underlying executable refers to a non-existent type declaration

MalformedParameterizedTypeException - if any of the underlying executable's parameter types refer to a parameterized type that cannot be instantiated for any reason

**Since:**

1.5

## getExceptionTypes

```
public Class<?>[] getExceptionTypes()
```

Returns an array of `Class` objects that represent the types of exceptions declared to be thrown by the underlying executable represented by this object. Returns an array of length 0 if the executable declares no exceptions in its `throws` clause.

**Specified by:**

getExceptionTypes in class Executable

**Returns:**

the exception types declared as being thrown by the executable this object represents

## getGenericExceptionTypes

```
public Type[] getGenericExceptionTypes()
```

Returns an array of `Type` objects that represent the exceptions declared to be thrown by this executable object. Returns an array of length 0 if the underlying executable declares no exceptions in its `throws` clause.

If an exception type is a type variable or a parameterized type, it is created. Otherwise, it is resolved.

**Overrides:**

getGenericExceptionTypes in class Executable

**Returns:**

an array of Types that represent the exception types thrown by the underlying executable

**Throws:**

GenericSignatureFormatError - if the generic method signature does not conform to the format specified in *The Java Virtual Machine Specification*

TypeNotPresentException - if the underlying executable's `throws` clause refers to a non-existent type declaration

`MalformedParameterizedTypeException` - if the underlying executable's `throws` clause refers to a parameterized type that cannot be instantiated for any reason

**Since:**

1.5

## equals

```
public boolean equals(Object obj)
```

Compares this `Method` against the specified object. Returns true if the objects are the same. Two `Methods` are the same if they were declared by the same class and have the same name and formal parameter types and return type.

**Overrides:**

`equals` in class `Object`

**Parameters:**

`obj` - the reference object with which to compare.

**Returns:**

`true` if this object is the same as the obj argument; `false` otherwise.

**See Also:**

`Object.hashCode()`, `HashMap`

## hashCode

```
public int hashCode()
```

Returns a hashcode for this `Method`. The hashcode is computed as the exclusive-or of the hashcodes for the underlying method's declaring class name and the method's name.

**Overrides:**

`hashCode` in class `Object`

**Returns:**

a hash code value for this object

**See Also:**

`Object.equals(java.lang.Object)`, `System.identityHashCode(Object)`

## toString

```
public String toString()
```

Returns a string describing this `Method`. The string is formatted as the method access modifiers, if any, followed by the method return type, followed by a space, followed by the class declaring the method, followed by a period, followed by the method name, followed by a parenthesized, comma-separated list of the method's formal parameter types. If the method throws checked exceptions, the parameter list is followed by a space, followed by the word "`throws`" followed by a comma-separated list of the thrown exception types. For example:

```
public boolean java.lang.Object.equals(java.lang.Object)
```

The access modifiers are placed in canonical order as specified by "The Java Language Specification". This is `public`, `protected` or `private` first, and then other modifiers in the following order: `abstract`, `default`, `static`, `final`, `synchronized`, `native`, `strictfp`.

**Overrides:**

`toString` in class `Object`

**Returns:**

a string describing this `Method`

**See *Java Language Specification*:**

8.4.3 Method Modifiers
9.4 Method Declarations
9.6.1 Annotation Interface Elements

## toGenericString

```
public String toGenericString()
```

Returns a string describing this `Method`, including type parameters. The string is formatted as the method access modifiers, if any, followed by an angle-bracketed comma-separated list of the method's type parameters, if any, including informative bounds of the type parameters, if any, followed by the method's generic return type, followed by a space, followed by the class declaring the method, followed by a period, followed by the method name, followed by a parenthesized, comma-separated list of the method's generic formal parameter types. If this method was declared to take a variable number of arguments, instead of denoting the last parameter as "*Type*`[]`", it is denoted as "*Type*`...`". A space is used to separate access modifiers from one another and from the type parameters or return type. If there are no type parameters, the type parameter list is elided; if the type parameter list is present, a space separates the list from the class name. If the method is declared to throw exceptions, the parameter list is followed by a space,

followed by the word "throws" followed by a comma-separated list of the generic thrown exception types.

The access modifiers are placed in canonical order as specified by "The Java Language Specification". This is public, protected or private first, and then other modifiers in the following order: abstract, default, static, final, synchronized, native, strictfp.

**Overrides:**

toGenericString in class Executable

**Returns:**

a string describing this Method, include type parameters

**See *Java Language Specification*:**

8.4.3 Method Modifiers
9.4 Method Declarations
9.6.1 Annotation Interface Elements

**Since:**

1.5

## invoke

```
public Object invoke(Object obj,
                     Object... args)
              throws IllegalAccessException,
                     InvocationTargetException
```

Invokes the underlying method represented by this Method object, on the specified object with the specified parameters. Individual parameters are automatically unwrapped to match primitive formal parameters, and both primitive and reference parameters are subject to method invocation conversions as necessary.

If the underlying method is static, then the specified obj argument is ignored. It may be null.

If the number of formal parameters required by the underlying method is 0, the supplied args array may be of length 0 or null.

If the underlying method is an instance method, it is invoked using dynamic method lookup as documented in The Java Language Specification, section §15.12.4.4; in particular, overriding based on the runtime type of the target object may occur.

If the underlying method is static, the class that declared the method is initialized if it has not already been initialized.

If the method completes normally, the value it returns is returned to the caller of invoke; if the value has a primitive type, it is first appropriately wrapped in an object. However, if the value has the type of an array of a primitive type, the elements of the array are *not* wrapped in objects; in other words, an array of primitive type is returned. If the underlying method return type is void, the invocation returns null.

**Parameters:**

`obj` - the object the underlying method is invoked from

`args` - the arguments used for the method call

**Returns:**

the result of dispatching the method represented by this object on `obj` with parameters `args`

**Throws:**

`IllegalAccessException` - if this `Method` object is enforcing Java language access control and the underlying method is inaccessible.

`IllegalArgumentException` - if the method is an instance method and the specified object argument is not an instance of the class or interface declaring the underlying method (or of a subclass or implementor thereof); if the number of actual and formal parameters differ; if an unwrapping conversion for primitive arguments fails; or if, after possible unwrapping, a parameter value cannot be converted to the corresponding formal parameter type by a method invocation conversion.

`InvocationTargetException` - if the underlying method throws an exception.

`NullPointerException` - if the specified object is null and the method is an instance method.

`ExceptionInInitializerError` - if the initialization provoked by this method fails.

## isBridge

```
public boolean isBridge()
```

Returns `true` if this method is a bridge method; returns `false` otherwise.

**API Note:**

A bridge method is a synthetic method created by a Java compiler alongside a method originating from the source code. Bridge methods are used by Java compilers in various circumstances to span differences in Java programming language semantics and JVM semantics.

One example use of bridge methods is as a technique for a Java compiler to support *covariant overrides*, where a subclass overrides a method and gives the new method a more specific return type than the method in the superclass. While the Java language specification forbids a class declaring two methods with the same parameter types but a different return type, the virtual machine does not. A common case where covariant overrides are used is for a `Cloneable` class where the `clone` method inherited from `java.lang.Object` is overridden and declared to return the type of the class. For example, `Object` declares

```
protected Object clone() throws CloneNotSupportedException {...}
```

and `EnumSet<E>` declares its language-level covariant override

```
public EnumSet<E> clone() {...}
```

If this technique was being used, the resulting class file for `EnumSet` would have two `clone` methods, one returning `EnumSet<E>` and the second a bridge method returning `Object`. The bridge method is a JVM-level override of `Object.clone()`. The body of the `clone` bridge method calls its non-bridge counterpart and returns its result.

**Returns:**

`true` if this method is a bridge method; returns `false` otherwise

**See *Java Language Specification*:**

8.4.8.3 Requirements in Overriding and Hiding
15.12.4.5 Create Frame, Synchronize, Transfer Control

**See *Java Virtual Machine Specification*:**

4.6 Methods

**Since:**

1.5

**See Also:**

Java programming language and JVM modeling in core reflection

## isVarArgs

```
public boolean isVarArgs()
```

Returns `true` if this executable was declared to take a variable number of arguments; returns `false` otherwise.

**Overrides:**

`isVarArgs` in class `Executable`

**Returns:**

`true` if this executable was declared to take a variable number of arguments; returns `false` otherwise

**See *Java Language Specification*:**

8.4.1 Formal Parameters

**Since:**

1.5

## isSynthetic

```
public boolean isSynthetic()
```

Returns `true` if this executable is a synthetic construct; returns `false` otherwise.

**Specified by:**

`isSynthetic` in interface `Member`

**Overrides:**

`isSynthetic` in class `Executable`

**Returns:**

true if and only if this executable is a synthetic construct as defined by *The Java Language Specification*.

**See *Java Language Specification*:**

13.1 The Form of a Binary

**See *Java Virtual Machine Specification*:**

4.6 Methods

**Since:**

1.5

**See Also:**

Java programming language and JVM modeling in core reflection

## isDefault

```
public boolean isDefault()
```

Returns `true` if this method is a default method; returns `false` otherwise. A default method is a public non-abstract instance method, that is, a non-static method with a body, declared in an interface.

**Returns:**

true if and only if this method is a default method as defined by the Java Language Specification.

**See *Java Language Specification*:**

9.4 Method Declarations

**Since:**

1.8

## getDefaultValue

```
public Object getDefaultValue()
```

Returns the default value for the annotation member represented by this `Method` instance. If the member is of a primitive type, an instance of the corresponding wrapper type is returned. Returns null if no default is associated with the member, or if the method instance does not represent a declared member of an annotation type.

**Returns:**

the default value for the annotation member represented by this `Method` instance.

**Throws:**

`TypeNotPresentException` - if the annotation is of type `Class` and no definition can be found for the default class value.

**See *Java Language Specification*:**

9.6.2 Defaults for Annotation Interface Elements

**Since:**

1.5

## getAnnotation

```
public <T extends Annotation> T getAnnotation
(Class<T> annotationClass)
```

Returns this element's annotation for the specified type if such an annotation is *present*, else null.

Note that any annotation returned by this method is a declaration annotation.

**Specified by:**

`getAnnotation` in interface `AnnotatedElement`

**Overrides:**

`getAnnotation` in class `Executable`

**Type Parameters:**

`T` - the type of the annotation to query for and return if present

**Parameters:**

`annotationClass` - the Class object corresponding to the annotation type

**Returns:**

this element's annotation for the specified annotation type if present on this element, else null

**Throws:**

`NullPointerException` - if the given annotation class is null

**Since:**

1.5

## getDeclaredAnnotations

```
public Annotation[] getDeclaredAnnotations()
```

Returns annotations that are *directly present* on this element. This method ignores inherited annotations. If there are no annotations *directly present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Note that any annotations returned by this method are declaration annotations.

**Specified by:**

`getDeclaredAnnotations` in interface `AnnotatedElement`

**Overrides:**

`getDeclaredAnnotations` in class `AccessibleObject`

**Returns:**

annotations directly present on this element

**Since:**

1.5

## getParameterAnnotations

```
public Annotation[][] getParameterAnnotations()
```

Returns an array of arrays of `Annotations` that represent the annotations on the formal parameters, in declaration order, of the `Executable` represented by this object. Synthetic and mandated parameters (see explanation below), such as the outer "this" parameter to an inner class constructor will be represented in the returned array. If the executable has no parameters (meaning no formal, no synthetic, and no mandated parameters), a zero-length array will be returned. If the `Executable` has one or more parameters, a nested array of length zero is returned for each parameter with no annotations. The

annotation objects contained in the returned arrays are serializable. The caller of this method is free to modify the returned arrays; it will have no effect on the arrays returned to other callers. A compiler may add extra parameters that are implicitly declared in source ("mandated"), as well as parameters that are neither implicitly nor explicitly declared in source ("synthetic") to the parameter list for a method. See `Parameter` for more information.

Note that any annotations returned by this method are declaration annotations.

**Specified by:**

`getParameterAnnotations` in class `Executable`

**Returns:**

an array of arrays that represent the annotations on the formal and implicit parameters, in declaration order, of the executable represented by this object

**Since:**

1.5

**See Also:**

`Parameter`, `Parameter.getAnnotations()`

## getAnnotatedReturnType

```
public AnnotatedType getAnnotatedReturnType()
```

Returns an `AnnotatedType` object that represents the use of a type to specify the return type of the method/constructor represented by this Executable. If this `Executable` object represents a constructor, the `AnnotatedType` object represents the type of the constructed object. If this `Executable` object represents a method, the `AnnotatedType` object represents the use of a type to specify the return type of the method.

**Specified by:**

`getAnnotatedReturnType` in class `Executable`

**Returns:**

an object representing the return type of the method or constructor represented by this `Executable`

**Since:**

1.8

---

contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.

Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.

All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.